```
     * Handle the Employee “restored” event.
     */
    public function restored(User $employee): void
    {
     // Code Logic
    }
    /**
     * Handle the Employee “forceDeleted” event.
     */
    public function forceDeleted(User $employee): void
    {
     // Code Logic
    }
}
```

At times, you might need to **mute** a model’s whole firing of events. This can be achieved by utilizing the **`withoutEvents`** technique. The only input for the **`withoutEvents`** function is a closure. The **`withoutEvents`** function will return any result returned by the closure, and any code performed within this closure won’t dispatch model events.

Conclusion

In this chapter, *Working with Eloquent ORM and Query Builder*, the focus was on exploring the fundamental concepts of Eloquent ORM and Query Builder in the context of Laravel, a popular PHP framework for web development. The chapter began by introducing the importance of Eloquent and Query Builder in database operations within web development projects.

The chapter’s goal was to provide readers with a thorough grasp of Laravel’s Eloquent ORM and Query Builder while highlighting their significance and illuminating how to use them through real-world scenarios. It discussed key ideas and methods that programmers may use to create effective and dependable database-driven Laravel applications.

In the next chapter, we will look at the security aspects of the Laravel application, including how to incorporate authentication and authorization in Laravel.